



BITS Pilani
Pilani Campus

Deploying Microservices Contd.

Prof. Akanksha Bharadwaj
Asst. Professor, CSIS Department



BITS Pilani
Pilani Campus



SE ZG583, Scalable Services

Lecture No. 13

Agenda



Deploying microservices

- Deploying services without downtime: Canaries, Blue-Green, & rolling deploys
- Deploying microservices using Serverless deployment

Deployment with Containers

- Introduction to containers
- Containerizing a service
- Deploying to a cluster

Introduction of deployment Strategies



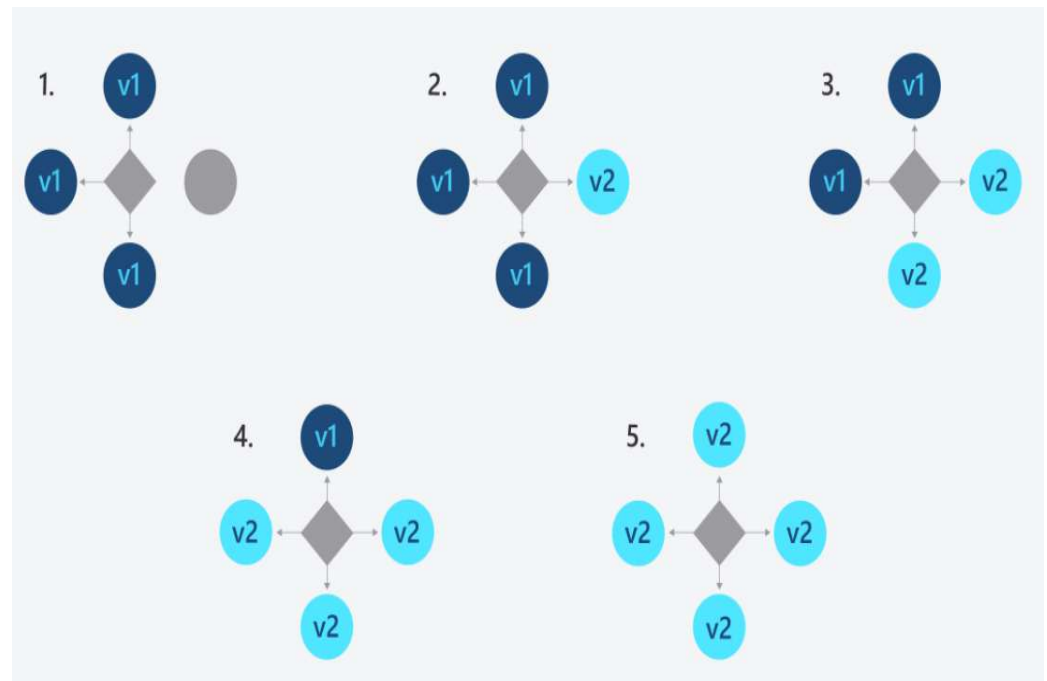
Here are four common deployment strategies that organizations use in production

- Ramped
- Blue/Green
- Canary
- A/B testing

Ramped



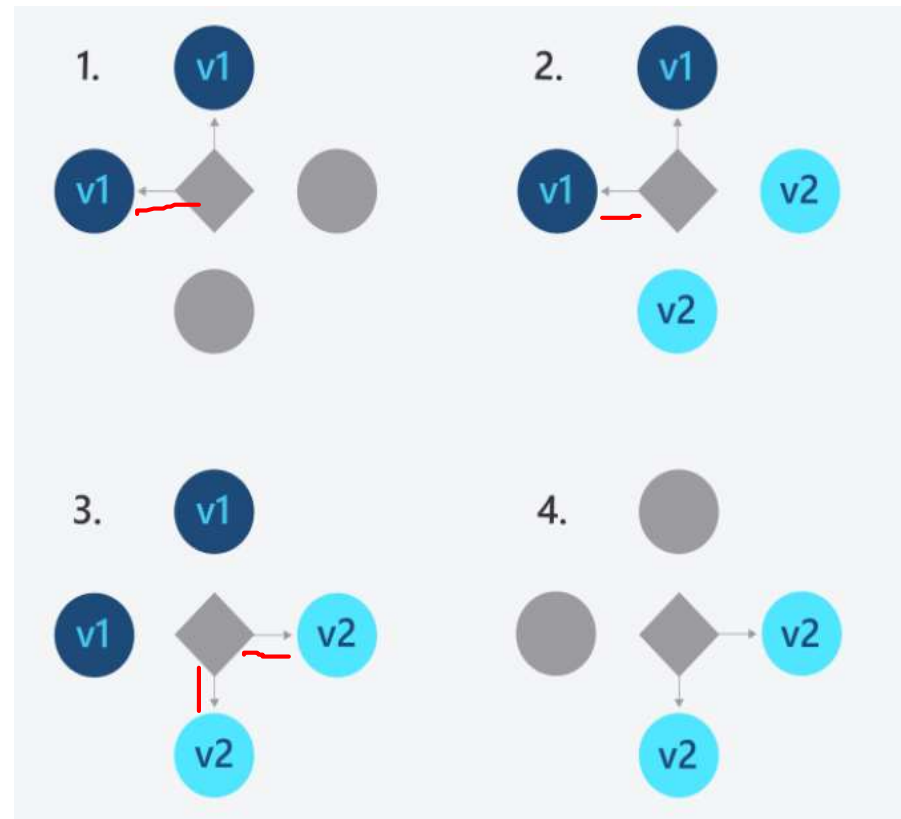
- This is the default roll-out method in Kubernetes.
- This deployment slowly replaces pods one at a time to avoid downtime.



Blue/Green



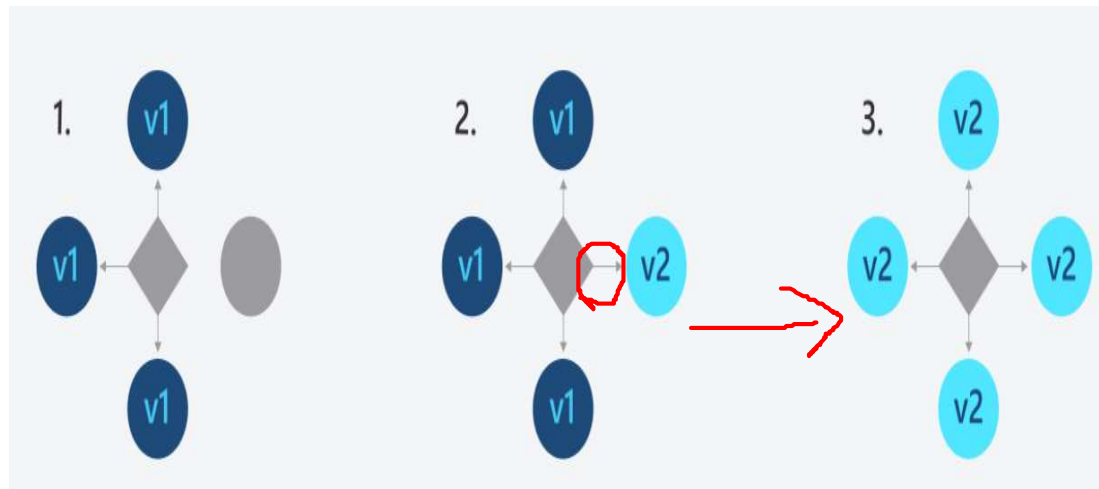
- In this you release a new version of your application or workflow while your current version is still running.



Canary



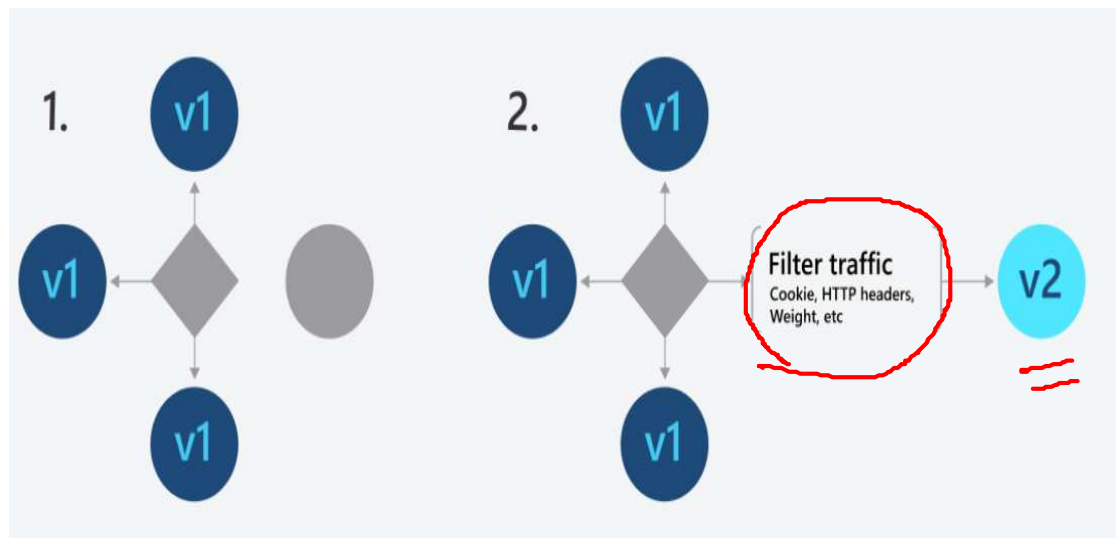
- Allow the customers to test your Kubernetes deployment by releasing the new version to a small group of them.



A/B testing



- Much like the canary Kubernetes deployment strategy, an A/B testing strategy targets a specific group of customers.



Serverless Deployment



- It is a deployment infrastructure that hides any concept of servers physical or virtual hosts, or containers.
- To deploy your service using this approach, you package the code (e.g. as a ZIP file), upload it to the deployment infrastructure and describe the desired performance characteristics.

Drawbacks of using Serverless Deployment



- It's not the best scenario for executing long-running applications.
- Vendor lock in
- Cold start

Examples of Serverless Deployment



- AWS Lambda
- Google Cloud Functions
- Azure Functions

Introduction to containers



- A container is a lightweight, portable, and self-contained unit of software that packages together an application and its dependencies, libraries, and configuration files, ensuring consistency and reliability across different computing environments.
- Containers provide a standardized runtime environment for applications, enabling them to run reliably and consistently on any platform that supports containerization.

Container Image



- A container image is a lightweight, standalone, executable package that contains everything needed to run a piece of software, including the application code, runtime, libraries, dependencies, and configuration files.

Docker Containers



- Docker containers are lightweight, portable, and self-contained units of software that encapsulate an application along with its dependencies, libraries, and configuration files.
- Docker is a containerization platform that allows developers to create, deploy, and manage containers easily, providing a consistent runtime environment across different computing environments.
- Docker provides the ability to package and run an application in a loosely isolated environment called a container.

Key aspects of Docker containers



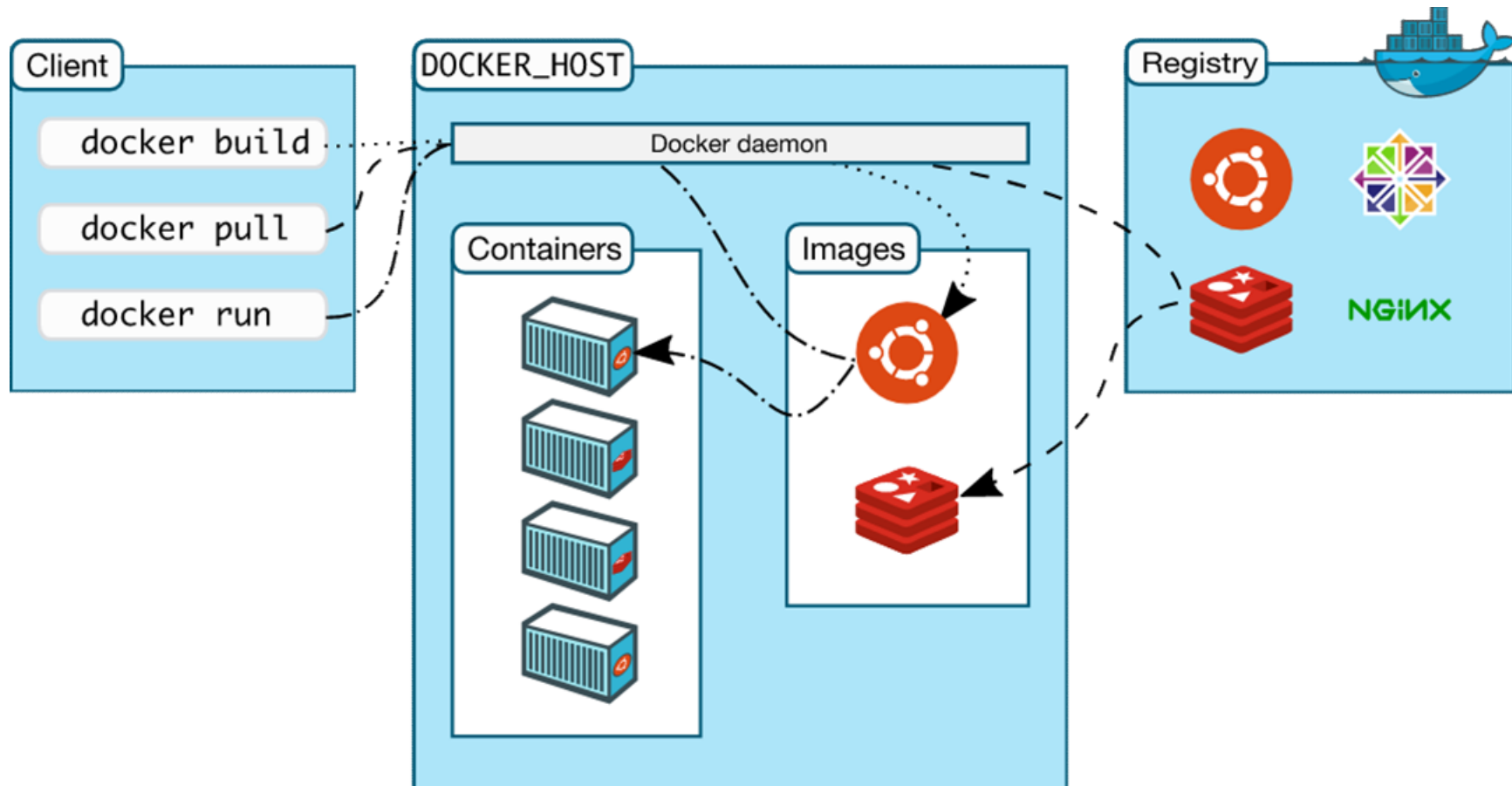
- Image-Based
- Isolation
- Portability
- Efficiency
- Orchestration
- Microservices
- DevOps

What can I use Docker for?



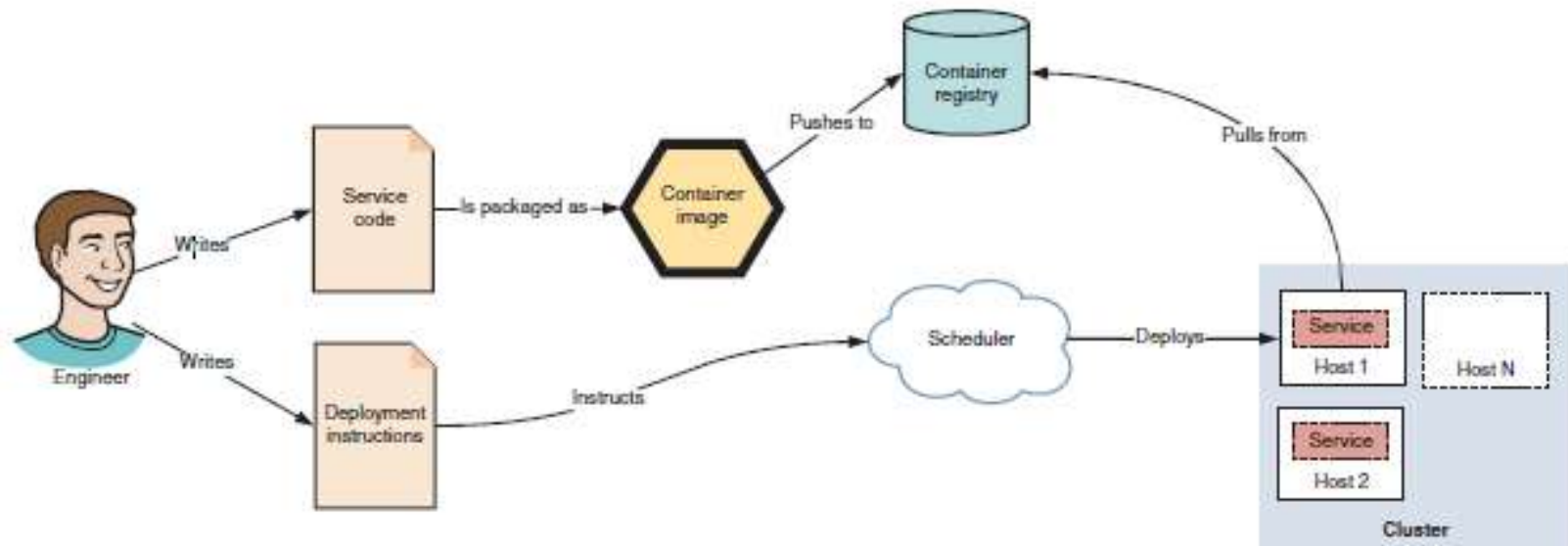
- Fast and consistent delivery of your applications
- Microservices Architecture
- Scalability

Docker architecture



Containerizing a service

- Build an image for a service
- Run multiple instances or containers of your image
- Push your image to a shared repository, or registry



Build and Run Docker Image



Steps required:

1. Install Docker
2. Build your Docker image
3. Run your Docker image

Working with images



- The image will include the file system that your application needs to run code and dependencies and other metadata, such as the command that starts your application.
- Example: **docker pull python:3.6**

Building your image



- To build image, first you need to create a Dockerfile
- Example: **docker build -t market-data:first-build .**

Running containers



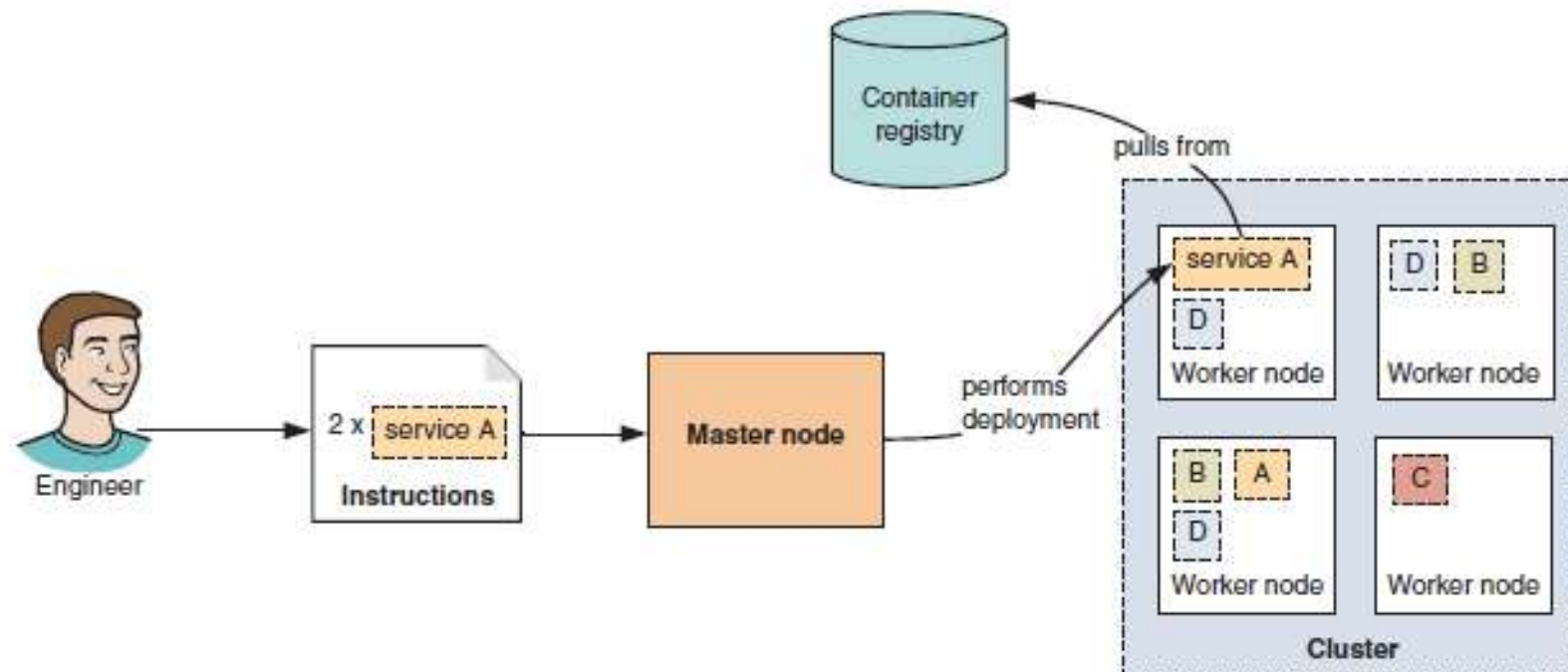
- Example:
- **`docker run -d -p 80:80 docker/getting-started`**
- **`docker run -d -p 8000:8000 --name market-data market-data:latest`**

Storing an image



- To push to this repository, you need to tag your image with an appropriate name.
- Docker image names follow the format `<registry>/<repository>:<tag>`.

Deploying to a cluster



High-level scheduler architecture and deployment process

Update the application

Steps

1. Update the source code
2. Build our updated version of the image, using the same command we used before.

docker build -t getting-started:v2 .

3. Start a new container using the updated code (later deleting old container)

docker run -dp 3000:3000 getting-started:v2

Share the application



Create a repo

1. Sign in to Docker Hub.
2. Click the Create Repository button.
3. Give the desired repo name and make sure the Visibility is Public.
4. Click the Create button!

Play with Docker



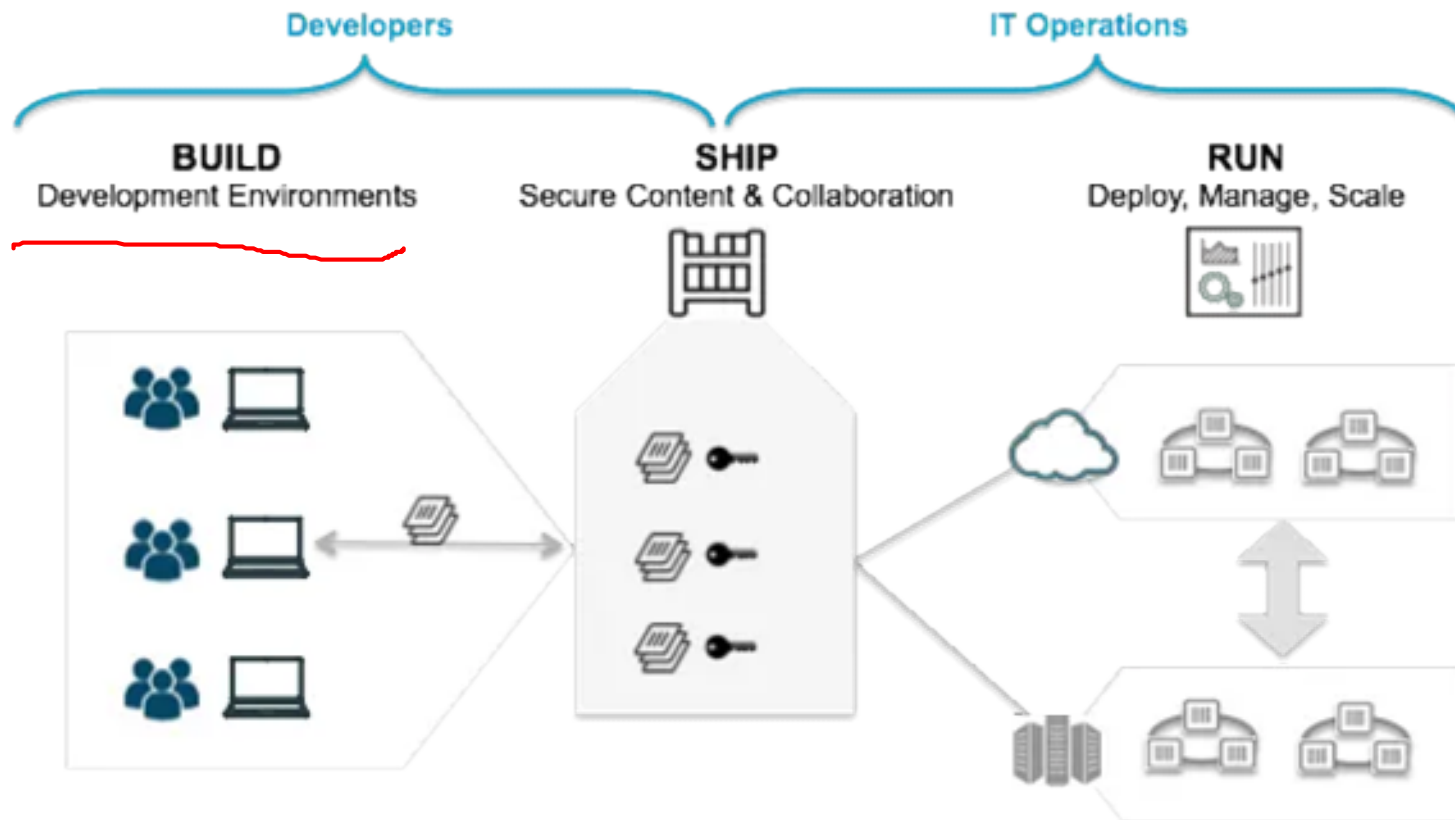
1. Open your browser to Play with Docker:
<https://labs.play-with-docker.com/>
2. <https://training.play-with-docker.com/>

Containers as a Service (CaaS)



- Containers as a Service is a model where IT organizations and developers can work together to build, ship and run their applications anywhere.
- CaaS enables an IT secured and managed application environment consisting of content and infrastructure, from which developers are able to build and deploy applications in a self service manner.

CaaS workflow



Docker and git



Ques. How to use GitHub with Docker

Self study:

Link: <https://docs.docker.com/ci-cd/github-actions/>

Self Study



- <https://aws.amazon.com/blogs/apn/deploying-code-faster-with-serverless-framework-and-aws-service-catalog/>
- How to use GitHub with Docker: <https://docs.docker.com/ci-cd/github-actions/>
- Minikube: <https://minikube.sigs.k8s.io/docs/start/>

References



- Chapter 8,9 Microservices in Action
- Link: <https://azure.microsoft.com/en-in/overview/kubernetes-deployment-strategy/>
- <https://www.docker.com/blog/containers-as-a-service-caas/>
- <https://docs.docker.com/>